

数据结构

CoreLoop

写给读者

本书假设大家已经有了比较好的 C 语言基础，但作者也会在某些合适的地方强调某些 C 语言的知识点。

如果你对本书中的内容存在疑问，欢迎联系作者。**【联系方式】**

许多地方添加一些内容，能够对正文有很好的补充及/或帮助大家理解的作用，但是为了保证在本书中不过分多地引入与《数据结构》本身无关的内容，作者并未给出这些“DLC”。如果想要了解，请参考我的视频。

重要提示：本书出现的所有“思维拓展”块都“可！以！完！全！跳！过！”，作者保证读者不会错过任何《数据结构》本身的内容。但是从逻辑上说，“可以”跳过，换句话说就是“可以”不跳过。大家可以根据最适合自己的方式选择是否阅读。笔者给大家的建议是：先看一遍，如果看懂了，相信会是很大的收获；如果没看懂，立即跳过。

知识绝不会少，但更重要的是思考。

很多有意思的东西都是藏在习题里面。

目录

第一章 树与二叉树	1
1.1 树	1
1.1.1 * 目录树的简单实现	3
1.1.2 树的一些基本概念	5
1.2 二叉树	5
习题参考答案	7

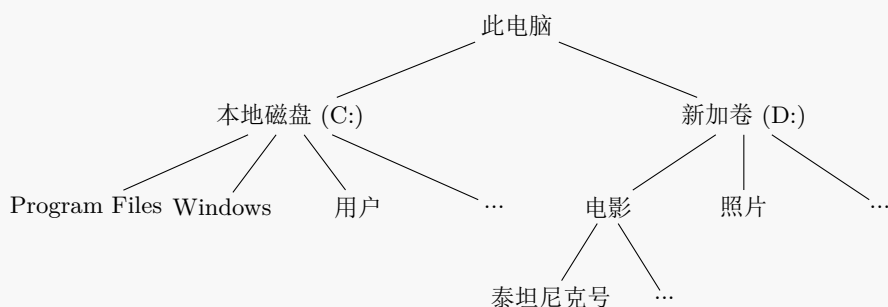
第一章 树与二叉树

1.1 树

在 ?? 节中，我们已经对“什么是树”有了一个感性的认知。与现实中的大树类似，数据结构中的树也是一种由根开始分叉生长的结构。比如游戏中一个队长三个队员的四人小队可以看成一棵树：



你的个人计算机中目录结构的组织是一棵树：



下面给出树的定义。

定义

树是 n ($n \geq 0$) 个数据元素组成的有限集合，当 $n=0$ 时，称为**空树**。任何一棵非空树都满足以下条件：

1. 有且仅有一个数据元素无前驱，称为**根节点**；
2. 除根节点外，其余每个数据元素都有且仅有一个前驱节点，称为**父节点**；
3. 每个数据元素可以有零个或多个后继节点，称为**子节点**。

由以上定义可知，对于一棵非空树中的任意一个节点，以该节点为根节点，连同该节点的所有后代节点在内所构成的仍是一棵树，称为原树的一棵**子树**。通常认为一棵树本身是自己的子树。很少存在一些场景会关心树中节点的各个子树从左至右是否有次序（即互换后还能不能认为是同一棵树），但通常将节点有次序的树称为**有序树**，节点没有次序的树称为**无序树**。

“树”这个逻辑结构该如何实现？如果采用链表类似的思想，每个节点除保存数据外，再保存所

有指向其孩子节点的指针，像这样：

```
1 struct TreeNode {
2     ElemType data;
3     TreeNode* child1;
4     TreeNode* child2;
5     TreeNode* child3;
6     ...
7 }
```

显然，我们无法预知每个节点有多少孩子节点，因此无法为 `TreeNode` 结构定义固定数量的孩子节点指针。熟悉 C++ 的同学可能会想到，可以使用一个不定长数组，将节点的所有孩子指针保存到该数组中：

```
1 struct TreeNode {
2     ElemType data;
3     std::vector<TreeNode*> childs; // childs 是一个不定长数组，childs[0], ..., childs[n] 分别表示该节点的
    ↪ 各个孩子节点指针
4 }
```

如图 1.1 所示。这是完全可行，也非常自然的一种树的表示方式。但本书不要求大家熟悉 C++，

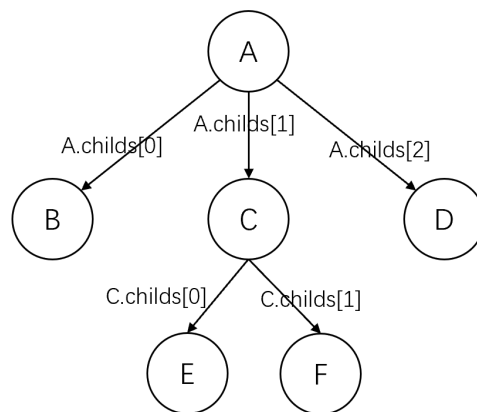


图 1.1 用不定长数组表示树的孩子

因此我们给出树的另一种表示方法，也是更高效的孩子-兄弟表示法（**First Child - Next Sibling Representation**）。该方法存储每个节点的第一个孩子的指针，以及下一个兄弟的指针：

```
1 struct TreeNode {
2     ElemType data;
3     TreeNode* first_child;
4     TreeNode* next_sibling;
5 };
```

如图 1.2 所示。

显然该表示法下，只要拿到根节点指针，就能找到树中任何一个节点。因此，可以用根节点指针表示一棵树。如：

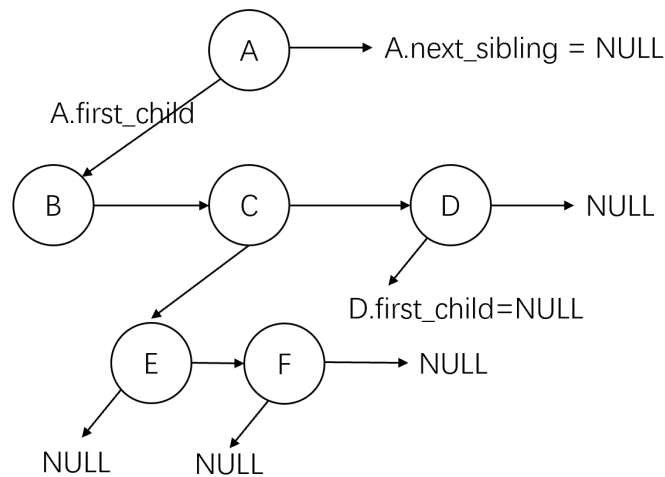


图 1.2 树的孩子-兄弟表示法

```
TreeNode* root;
```

root 这个对象既可以理解为树的根节点的指针，又可以理解为以 root 节点为根的一棵树。

提示

图 1.1 与图 1.2 逻辑上是同一棵树，只不过物理指针的含义不同。

有了以上的内容介绍，我们来做一点有意思的事情（当然你大可以放心地跳过）：用孩子-兄弟表示法尝试实现操作系统的目录树。

1.1.1 * 目录树的简单实现

我们已经知道，操作系统对目录的管理其实就是一棵树。用户随时可能在任何目录新建目录、删除某个目录，我们暂且不考虑文件。并假设目录节点的“数据”只有一个字符串类型的成员，表示目录的名字，如“电影”等。以下代码实现了简单的新建目录、删除目录的操作，具体过程请结合注释。

```

1 // 在目录 Dad 中，新建一个名为 name 的目录
2 void DirInsert(TreeNode* Dad, char* name) {
3     // 函数实现：创建一个新节点，将其加入 Dad 的孩子链表末尾
4     TreeNode* new_node = (TreeNode*)malloc(sizeof(TreeNode));
5     strcpy(new_node->name, name);
6     new_node->first_child = NULL;
7     new_node->next_sibling = NULL;
8
9     // Dad 原本是空目录，则需要更改 first_child 指针。
10    if(Dad->first_child == NULL) {
11        Dad->first_child = new_node;
12        return;
13    }
14
15    // 原本不是空目录，修改 next_sibling 指针。

```

```
16     TreeNode* last_child = Dad->first_child;
17     while(last_child->next_sibling != NULL) {
18         last_child = last_child->next_sibling;
19     }
20     last_child->next_sibling = new_node;
21     return;
22 }
23
24 // 销毁一个节点及其所有后代
25 void DestroyTree(TreeNode* root) {
26     if(root == NULL) return;
27     // 先递归销毁其子树
28     DestroyTree(root->first_child);
29
30     // 再释放根节点
31     free(root);
32 }
33
34 // 在目录 Dad 中，删除目录 to_del
35 void DirDelete(TreeNode* Dad, TreeNode* to_del) {
36     // 函数实现：先把 to_del 从父节点的孩子链表中摘掉，再销毁以 to_del 为根的整棵树
37     if(Dad == NULL) {
38         return;
39     }
40     // 要删除的就是第一个孩子节点，需要更新 first_child 指针
41     if(to_del == Dad->first_child) {
42         Dad->first_child = to_del->next_sibling;;
43         // 注意这里不能简单地 free(to_del)，因为该目录中可能还有若干目录，目录中可能还有目录，因此要
44         ↪ 递归销毁。
45         DestroyTree(to_del);
46         return;
47     }
48     // 找到要删除的节点的前一个兄弟节点
49     TreeNode* pre = Dad->first_child;
50     while(pre->next_sibling != to_del) {
51         pre = pre->next_sibling;
52     }
53
54     // 修改链条，类似单链表的删除操作
55     pre->next_sibling = to_del->next_sibling;
56     // 注意同样要递归销毁
57     DestroyTree(to_del);
58     return;
59 }
```

提示

以上例子也给了我们一个重要提示：树中每个节点的子节点很多时候不能单纯的看成一个节点，而是要看成一棵子树。因此与树相关的操作，很多都会涉及到递归，这一点非常重要。

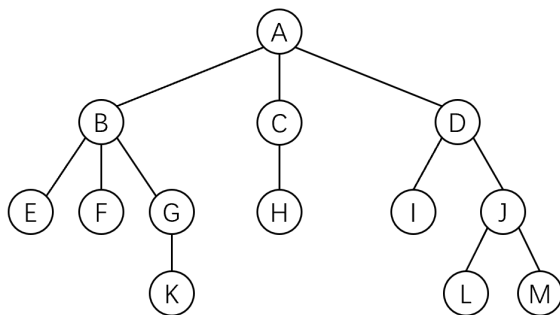


图 1.3

1.1.2 树的一些基本概念

我们回到逻辑上的数据结构：树。树中有一些大家常用的专业术语与性质，我们在这里列出。

a) 树中，一个节点拥有的子树数量称为**节点的度**。一棵树中所有节点的度的最大值称为这棵树的**度**。

b) 树中度为 0 的节点称为**叶子节点**，简称**叶节点**。显然叶节点没有孩子节点。不是叶子节点的节点称为**非叶节点**。

图 1.3 中，节点 A、B 的度均为 3，整棵树的度为 3；节点 E、F、K、H、I、L、M 的度为 0，是叶子节点。

c) 可以看出树是一种具有层次结构的数据结构。根节点处于最高层，通常定义根为第 1 层，根的孩子为第 2 层。若某节点处在第 i 层，其子树的根节点就处在第 $i+1$ 层。

d) **节点的深度**通常定义为节点的所在层次；而**节点的高度**通常是指从最底层叶子节点（高度为 1）开始逐层累加至该节点时的高度值。所有节点的深度的最大值称为**树的深度**或**树的高度**。即

$$\text{树的高度} = \max\{\text{所有节点的深度值}\} = \max\{\text{所有节点的高度值}\}$$

图 1.3 中，节点 A 的深度为 1，高度为 4；节点 B 的深度为 2，高度为 3；节点 K 的深度为 4，高度为 1。整棵树的深度，或称整棵树的高度，为 4。

提示

1. 有些人喜欢将根节点的高度定义为 0，本书若无特别说明，默认按照根节点高度为 1 来定义。
2. 注意区分节点与树的深度和高度：节点的高度和节点的深度通常不是一回事；而树的高度和树的深度一定是相等的。

1.2 二叉树

很少有实际问题需要直接通过二叉树来解决，但是二叉树在计算机科学中的应用是非常非常重要的。许多更加有用的数据结构都是一棵二叉树。而且对二叉树的研究可以为我们建立类似树的层次结构的思维方式。笔者认为，对“二叉树”的研究在整个《数据结构》中都是相当重要的。

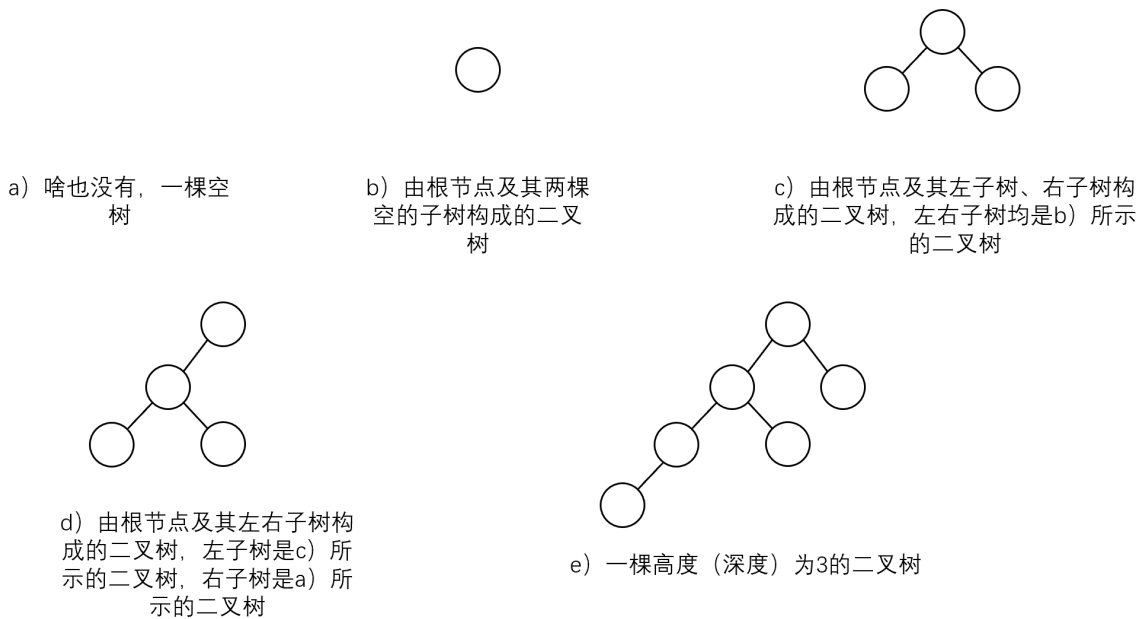


图 1.4 几棵不同的二叉树

定义

二叉树要么是一棵空树，要么是一棵：

由根节点以及它的两棵子树构成的树，这两棵子树有左右位置之分，分别称为左子树和右子树，并且左子树和右子树依然是一棵二叉树。

可以看到，二叉树是递归定义的，即用二叉树自身来定义二叉树。如图 1.4 所示的几棵树都是二叉树，根据图中文字介绍，大家也能进一步体会二叉树的递归定义。

显然二叉树的度只可能是 0、1 或 2，二叉树中每一个节点的度也只可能是 0、1 或 2。

提示

“二叉树”与“度为 2 的有序树”有如下区别：

1. 度为 2 的树至少有 3 个节点，而二叉树未必；
2. 二叉树即使只有一个孩子节点，该节点也要区分左右。而度为 2 的有序树，其“有序”是兄弟之间的位置有序，若某个节点是“独生子女”，则其位置是唯一确定的，不分左右。二叉树明确区分孩子的左右位置，这是因为在更多的场景下这是有用的，在后面的内容中我们将有所体会。

习题参考答案